Отчёт по индивидуальному заданию №1

Моделирование работы супермаркета

Исполнитель: Батышкина Алёна Игоревна

Группа: БПИ253

Вариант: 8

Дата: 07.10.2026

---

1. Условие задачи

Разработать последовательную программу на языке C, моделирующую процесс обслуживания покупателей в супермаркете. Программа должна использовать событийно-ориентированный подход, обеспечивать наглядное представление результатов в реальном времени и корректно завершаться при любых входных данных.

Ключевые сущности предметной области:

- Покупатели, прибывающие в супермаркет в случайные моменты времени;
- Кассы с очередями ограниченной вместимости;
- Кассиры, обслуживающие покупателей.

Параметры модели задаются через конфигурационный файл и включают количество покупателей, вместимость очередей, временные интервалы, стратегию выбора очереди и продолжительность рабочего дня.

---

2. Сценарий моделирования

2.1. Отображение сущностей в процессы

В программе принята событийно-ориентированная архитектура. Каждая сущность предметной области представлена структурой данных, а её поведение — обработчиками событий.

| Сущность предметной области | Представление в программе | Поведение |
|=============================|===========================|===========| | Покупатель | структура `Customer` (id, количество товаров, время прибытия, время ожидания) | Прибытие, Выбор товаров, Выбор очереди, Ожидание, Обслуживание или уход | | Кассир | структура `Cashier` (id, состояние сна, текущий покупатель, статистика) | Сон, Пробуждение,

Обслуживание, Сон | | Очередь | структура `Queue` (массив ID покупателей, вместимость, размер) | Добавление покупателя, Удаление покупателя | | Событие | структура `Event` (тип, время, идентификатор сущности) | Извлечение из min-кучи, Обработка, Генерация новых событий |

2.2. Жизненный цикл покупателя

1. **CUSTOMER_ARRIVE** — покупатель прибывает в супермаркет, генерируется случайное количество товаров и время завершения выбора.
2. **FINISH_SHOPPING** — покупатель завершает выбор товаров и переходит к выбору очереди.
3. **QUEUE_CHOICE** — покупатель выбирает очередь согласно заданной стратегии (SHORTEST, FIRST или RANDOM). Если очередь переполнена, генерируется событие ухода.
4. **LEAVE_NO_SERVICE** — покупатель уходит без обслуживания (очередь заполнена).
5. **CASHIER_WAKE** — кассир пробуждается, если очередь была пуста.
6. **START_CHECKOUT** — начинается обслуживание покупателя.
7. **FINISH_CHECKOUT** — обслуживание завершено, кассир переходит к следующему покупателю или засыпает.

2.3. Взаимодействие процессов

Все события хранятся в min-куче (`EventQueue`), отсортированной по времени возникновения. При совпадении времени события обрабатываются в порядке приоритета: завершения обслуживания имеют высший приоритет, прибытия новых покупателей — низший. Это обеспечивает корректную обработку «одновременных» событий.

---

3. Архитектура программы

3.1. Структура проекта

```
project/
├── include/ Заголовочные файлы
│ ├── types.h Базовые типы и структуры
│ ├── config.h Интерфейс конфигурации
│ ├── event_queue.h Интерфейс очереди событий
│ ├── io.h Интерфейс ввода-вывода
│ ├── rng.h Интерфейс генератора случайных чисел
│ ├── simulation.h Интерфейс симуляции
│ ├── events.h Интерфейс обработчиков событий
│ └── display.h Интерфейс визуализации
├── src/ Исходные файлы
│ ├── main.c Точка входа
│ ├── config.c Загрузка и валидация конфигурации
│ ├── event_queue.c Реализация min-кучи
│ ├── io.c Ввод-вывод через системные вызовы
│ ├── rng.c Генератор случайных чисел
```

```
| ├── simulation.c Логика симуляции
| ├── events.c Обработчики событий
| ├── display.c Табличный интерфейс
| └── types.c Массив имён событий
├── tests/ Тестовые конфигурации и скрипты
| ├── run_tests.sh Скрипт автоматического тестирования
| └── *.txt 18 тестовых конфигураций
├── Makefile Скрипт сборки
└── report.md Данный отчёт
```

3.2. Модульная структура

| Модуль | Ответственность | Используемые системные вызовы | |========|================-|-----------------------------| | `io.c` | Ввод-вывод в консоль и лог-файл | `open`, `write`, `close` | | `config.c` | Парсинг конфигурационного файла | `open`, `read`, `close` | | `rng.c` | Генерация случайных чисел | `srand`, `rand`, `time` | | `event_queue.c` | Min-куча событий | `calloc`, `free` | | `simulation.c` | Управление состоянием симуляции | `calloc`, `free`, `usleep` | | `events.c` | Обработчики событий | — | | `display.c` | Табличный интерфейс | `printf`, ANSI escape-коды | | `main.c` | Точка входа, обработка сигналов | `signal` |

---

4. Описание пользовательского интерфейса

4.1. Конфигурационный файл

Параметры задаются в текстовом файле в формате `ключ = значение`. Строки, начинающиеся с `#`, считаются комментариями. Пример:

```
total_customers = 50
queue_capacity = 5
shopping_time_min = 10
shopping_time_max = 30
arrival_interval_min = 2
arrival_interval_max = 5
items_min = 1
items_max = 8
item_service_time = 15
queue_strategy = 0
work_day_duration = 1800
random_seed = 42
display_delay_ms = 100
```

Параметр `queue_strategy` принимает значения: 0 — SHORTEST (кратчайшая очередь), 1 — FIRST (первая очередь), 2 — RANDOM (случайный выбор).

4.2. Табличный интерфейс в реальном времени

Во время выполнения симуляции в терминале отображается таблица, обновляющаяся на месте без прокрутки:

```
|============================================================|
| SUPERMARKET SIMULATION |
| Time: 0245 / 1800 |
|============================================================|
| Arrived: 28 Served: 12 Left: 16 |
|============================================================|
| Cashier 0: [C2][C5][C8] serving C2 |
| Cashier 1: [C3][C7] sleeping |
|============================================================|
| Recent events: |
| [0244] FINISH_CHECKOUT entity=0 |
| [0244] START_CHECKOUT entity=0 |
| [0240] CUSTOMER_ARRIVE entity=27 |
|============================================================|
```

Таблица содержит:

- Заголовок с текущим модельным временем и прогрессом рабочего дня;
- Сводную статистику (прибыло, обслужено, ушло);
- Состояние каждой кассы (список покупателей в очереди и статус кассира);
- Пять последних событий.

4.3. Лог-файл

Параллельно с табличным интерфейсом ведётся подробный лог в файле `simulation.log`. В лог записываются все события с временными метками и детальным описанием, а также итоговая статистика.

---

5. Варианты запуска и завершения программы

5.1. Способы запуска

Способ	Команда	Поведение
С конфигурационным файлом	`./supermarket config.txt`	Загрузка параметров из файла
Без параметров	`./supermarket`	Использование значений по умолчанию
С несуществующим файлом	`./supermarket missing.txt`	Сообщение об ошибке, использование значений по умолчанию
С невалидным файлом	`./supermarket invalid.txt`	Сообщение об ошибке, завершение с кодом 1

5.2. Способы завершения

Сценарий	Поведение

Сценарий	Поведение
Нормальное завершение	Все события обработаны, выводится статистика, проверяется закон сохранения, закрывается лог
Прерывание пользователем (Ctrl+C)	Перехват сигнала SIGINT, вывод статистики, освобождение ресурсов, закрытие лога
Ошибка конфигурации	Сообщение об ошибке, завершение с кодом возврата 1
Нарушение закона сохранения	Сообщение об ошибке, завершение с кодом возврата 1

---

6. Результаты работы программы

6.1. Пример запуска

При конфигурации: 50 покупателей, вместимость очереди 5, стратегия FIRST, seed = 42.

Итоговая статистика:

```
SIMULATION STATISTICS

Simulation duration: 1800 sec
Total arrived: 50
Total served: 6
Total left (no serv.): 44
Average wait time: 174.83 sec
Average shopping time: 19.46 sec
Max queue 0 size: 5
Max queue 1 size: 0
Cashier 0 served: 6
Cashier 1 served: 0

Conservation law verified: 50 = 6 + 44
```

6.2. Сравнение стратегий выбора очереди

Стратегия	Обслужено	Ушло без обслуживания	Эффективность
FIRST (все в первую очередь)	6	44	12%
SHORTEST (кратчайшая очередь)	~45	~5	~90%
RANDOM (случайный выбор)	~25	~25	~50%

Стратегия SHORTEST демонстрирует наилучшие показатели, так как равномерно распределяет нагрузку между кассами. Стратегия FIRST приводит к перегрузке первой очереди и массовому уходу покупателей.

6.3. Результаты тестирования

Автоматизированный тестовый набор включает 18 тестов:

- 13 позитивных тестов (различные конфигурации, граничные случаи, стресс-тесты);

- 5 негативных тестов (невалидные параметры, отсутствующий файл).

Все 18 тестов завершаются успешно. Закон сохранения покупателей соблюдается во всех сценариях.

---

7. Выводы

В ходе выполнения задания разработана событийно-ориентированная программа моделирования работы супермаркета на языке C. Реализованы следующие ключевые механизмы:

1. **Min-куча событий** с приоритизацией при совпадении времени;
2. **Три стратегии выбора очереди** с возможностью сравнения их эффективности;
3. **Табличный интерфейс** с обновлением в реальном времени через ANSI escape-последовательности;
4. **Разделение вывода** на консоль (таблица) и лог-файл (детальная информация);
5. **Ввод-вывод через системные вызовы** (`open` , `read` , `write` , `close`);
6. **Обработка прерываний** через сигнал SIGINT;
7. **Автоматизированное тестирование** с проверкой инвариантов.

Программа корректно завершается при любых входных данных, включая невалидные конфигурации и прерывание пользователем. Результаты моделирования подтверждают, что стратегия SHORTEST обеспечивает наибольшую пропускную способность супермаркета по сравнению со стратегиями FIRST и RANDOM.